FreeRTOS Core Features

1. Preemptive Multitasking

- FreeRTOS supports preemptive multitasking by default.
- Multiple tasks appear to execute simultaneously by sharing CPU time.
- The scheduler continuously monitors task priorities.
- If a higher-priority task becomes READY, the scheduler immediately switches CPU execution to that task.
- Context switching is performed automatically by the kernel.
- This mechanism ensures fast response to critical events.

Example

```
Task A Priority = 2 (Running)

Task B Priority = 4 becomes READY

Scheduler immediately switches CPU to Task B
```

Benefits

- Fast response to external events.
- Better CPU utilization.
- Suitable for real-time applications.

2. Priority-Based Scheduling

- Every task is assigned a priority when it is created.
- Priorities determine which task gets CPU time.
- Higher-priority tasks always execute before lower-priority tasks.
- Tasks with the same priority can share CPU time using Round Robin scheduling.
- Priority values are configurable through FreeRTOS configuration settings.

Example

```
Task A Priority = 3
Task B Priority = 2
Task C Priority = 1
```

Execution Order:

```
Task A
Task B
Task C
```

Benefits

- Critical tasks get immediate CPU access.
- Predictable task execution.
- Easy implementation of real-time systems.

3. Task Management

- A task is the basic unit of execution in FreeRTOS.
- Each task has its own:
  - Stack
  - Priority
  - Task Control Block (TCB)
- Scheduler allocates CPU time to tasks.
- Tasks can be dynamically created and deleted at runtime.

Common Operations

- Create Task
- Delete Task
- Suspend Task
- Resume Task
- Delay Task

Common APIs

```
xTaskCreate()
vTaskDelete()
vTaskSuspend()
vTaskResume()
vTaskDelay()
```

Applications

- Sensor reading task
- Communication task
- Display update task
- Data logging task

4. Inter-Task Communication (Queues)

- Queues are the most commonly used communication mechanism in FreeRTOS.

- They allow safe data transfer between tasks.
- Data is copied into the queue and retrieved by another task.
- Queue operations are thread-safe.
- Queues can also be accessed from ISRs using special APIs.

Example

```
Temperature Task
       |
       v
      Queue
       |
       v
Display Task
```

Common APIs

```
xQueueCreate()
xQueueSend()
xQueueReceive()
```

Advantages

- Safe communication between tasks.
- Prevents data corruption.
- Supports synchronization and data transfer.

5. Semaphores

- Semaphores are used for task synchronization.
- They allow one task or ISR to signal another task.
- A semaphore does not transfer data.
- It only indicates that an event has occurred.

Types

Binary Semaphore

- Value can be 0 or 1.
- Commonly used for ISR-to-task signaling.

Counting Semaphore

- Maintains a count.
- Useful when multiple events occur.

Example

```
ADC Conversion Complete
          |
          v
     Give Semaphore
          |
          v
 Processing Task Wakes Up
```

Benefits

- Efficient synchronization.
- Low CPU overhead.
- Fast ISR communication.

6. Mutex

- Mutex stands for Mutual Exclusion.
- Used to protect shared resources.
- Only one task can own a mutex at a time.
- Prevents race conditions and data corruption.
- Supports Priority Inheritance.

Example

```
Task A
    |
    v
 Shared UART
    ^
    |
Task B
```

Priority Inheritance

- Prevents Priority Inversion.
- Temporarily increases priority of mutex owner.

Common APIs

```
xSemaphoreCreateMutex()
xSemaphoreTake()
xSemaphoreGive()
```

7. Software Timers

- Software timers allow execution of functions after a specified delay.
- Implemented by FreeRTOS kernel.
- Do not consume dedicated hardware timers.
- Useful for periodic and one-shot operations.

Timer Types

One-Shot Timer

```
Runs once and stops.
```

Auto-Reload Timer

```
Runs periodically.
```

Applications

- LED blinking
- Sensor polling
- Timeout detection
- Periodic monitoring

Common APIs

```
xTimerCreate()
xTimerStart()
xTimerStop()
```

8. Memory Management

- FreeRTOS provides configurable memory allocation schemes.
- Used for dynamic creation of tasks, queues, semaphores, and timers.
- Different heap implementations allow optimization for memory usage and performance.

Available Heap Schemes

- heap_1 : Allocation only
- heap_2 : Allocation + Free
- heap_3 : Uses malloc/free
- heap_4 : Coalescing memory blocks
- heap_5 : Multiple memory regions

Most Common

```
heap_4.c
```

because it supports:

- Allocation
- Deallocation
- Memory block merging

Benefits

- Flexible memory management.
- Suitable for different embedded systems.

FreeRTOS Task Types

A task is the basic unit of execution in FreeRTOS. A task is similar to a thread in a general-purpose operating system. Each task has its own stack, priority, execution context, and Task Control Block (TCB).

Task Characteristics

Every FreeRTOS task contains:

- Task Control Block (TCB)
- Private Stack
- Priority
- Task State
- Task Function

Example:

```c
void SensorTask(void *pvParameters)
{
    while(1)
    {
        ReadSensor();
    }
}
```

Classification of Tasks

Tasks can be classified based on their execution behavior and priority requirements.

1. Periodic Tasks

Periodic tasks execute repeatedly at fixed intervals.

The task performs its operation and then waits until the next activation period.

Characteristics

- Fixed execution interval
- Predictable execution pattern
- Common in real-time systems

Example

```
Read Temperature Every 100 ms
```

Timeline:

```
0 ms    Read Sensor
100 ms  Read Sensor
200 ms  Read Sensor
300 ms  Read Sensor
```

APIs Used

```
vTaskDelay()
vTaskDelayUntil()
```

Applications

- Sensor Sampling
- Control Loops
- LED Blinking
- System Monitoring

2. Aperiodic Tasks

Aperiodic tasks execute only when a specific event occurs.

There is no fixed activation interval.

Characteristics

- Event-driven execution

- Remains blocked most of the time
- Consumes CPU only when required

Example

```
UART Data Received
        |
        v
Communication Task Executes
```

APIs Commonly Used

```
xQueueReceive()
xSemaphoreTake()
ulTaskNotifyTake()
```

Applications

- UART Reception
- Ethernet Packet Processing
- Button Press Detection
- External Interrupt Handling

---

3. Sporadic Tasks

Sporadic tasks are event-driven tasks with a minimum time gap between activations.

Unlike aperiodic tasks, their activation rate is controlled.

Characteristics

- Event-triggered
- Minimum inter-arrival time guaranteed
- Useful for overload protection

Example

```
Emergency Alarm
```

Requirement:

```
Maximum One Alarm Every 500 ms
```

Applications

- Emergency Shutdown Systems
- Alarm Monitoring
- Fault Detection

Task Types Based on Priority

FreeRTOS scheduling is priority based.

The highest priority READY task always gets CPU time.

High Priority Tasks

These tasks are time-critical.

Characteristics

- Short response time required
- Highest scheduling preference

Examples

- Motor Control
- Safety Functions
- Emergency Handling
- Critical Sensor Processing

Medium Priority Tasks

These tasks are important but not safety critical.

Examples

- Communication Protocols
- Data Processing
- Display Updates

Low Priority Tasks

These tasks perform background operations.

Examples

- Logging
- Diagnostics

- Statistics Collection
- Debug Information

Idle Task

The Idle Task is automatically created by FreeRTOS.

Characteristics

- Lowest priority task
- Always present
- Runs when no other task is READY

Responsibilities

- CPU Idle Processing
- Memory cleanup for deleted tasks
- Low-power management (optional)

Priority:

```
Priority = 0
```

Timer Service Task

The Timer Service Task is automatically created when software timers are enabled.

Responsibilities

- Execute timer callback functions
- Process timer commands
- Manage timer expirations

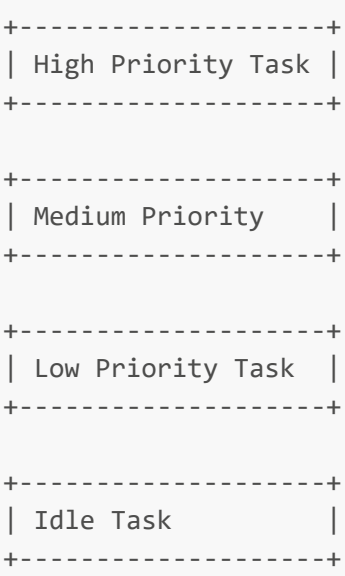
Example

```
Software Timer Expires
          |
          v
Timer Service Task
          |
          v
Callback Function Executes
```

Configuration

```
configUSE_TIMERS = 1
```

Typical Task Structure



Advantages of Task-Based Design

- Modular software design
- Easy maintenance
- Better scalability
- Improved responsiveness
- Simplified scheduling
- Efficient CPU utilization

Important Interview Questions

What is a Task in FreeRTOS?

A task is the basic unit of execution in FreeRTOS, similar to a thread in a traditional operating system.

What is the difference between Periodic and Aperiodic Tasks?

Periodic tasks execute at fixed intervals, whereas aperiodic tasks execute only when an event occurs.

What is the Idle Task?

The Idle Task is an automatically created low-priority task that executes when no other task is ready.

What is the Timer Service Task?

A FreeRTOS system task responsible for executing software timer callbacks and managing timer events.

Which task gets CPU time first in FreeRTOS?

The highest-priority READY task gets CPU time first.

SysTick Timer

- SysTick (System Timer) is a built-in 24-bit down-counter timer available in ARM Cortex-M processors.
- It is part of the Nested Vectored Interrupt Controller (NVIC). Primarily used to generate periodic interrupts for operating system scheduling and time management.
- Counts down from a programmed value to zero.
- When the counter reaches zero:
  - A SysTick Interrupt is generated.
  - The counter automatically reloads and starts counting again.

Uses of SysTick Timer

- RTOS scheduler tick generation
- Task scheduling
- Time delays
- System time keeping
- Periodic event generation

SysTick in FreeRTOS

- FreeRTOS typically uses SysTick as the system tick source.
- Generates periodic interrupts (usually every 1 ms).
- On each tick:
  - Tick count is incremented.
  - Delayed tasks are checked.
  - Scheduler decides if a context switch is required.

FreeRTOS Scheduling

Scheduling is the process of deciding which task should execute on the CPU at any given time.

The FreeRTOS scheduler is responsible for selecting tasks, performing context switching, and ensuring that high-priority tasks receive CPU time when required.

Why Scheduling is Required?

In an embedded system, multiple tasks may need CPU time simultaneously.

Examples:

- Sensor Reading Task
- Communication Task
- Display Task
- Logging Task

Since a CPU can execute only one task at a time on a single core, the scheduler determines which task should run.

FreeRTOS Scheduler

The scheduler is the heart of the FreeRTOS kernel.

Responsibilities:

- Maintain task states
- Maintain task priorities
- Select next task to execute
- Perform context switching
- Handle time slicing

The scheduler executes automatically and does not require direct intervention from the application.

Task States

Every FreeRTOS task exists in one of several states.

Running State

- Task currently executing on CPU.
- Only one task can be in Running state per CPU core.

```
CPU
 |
Running Task
```

Ready State

- Task is ready to execute.
- Waiting for CPU allocation.

```
Ready Queue
   |
```

```
Task A
Task B
Task C
```

Blocked State

Task is waiting for:

- Queue data
- Semaphore
- Mutex
- Notification
- Timeout expiration

Example:

```
xQueueReceive();
```

The task remains blocked until data becomes available.

Suspended State

Task has been explicitly suspended.

Example:

```
vTaskSuspend();
```

The scheduler ignores suspended tasks until resumed.

Priority-Based Scheduling

FreeRTOS uses priority-based scheduling.

Every task is assigned a priority.

Example:

```
Task A Priority = 5
Task B Priority = 3
Task C Priority = 1
```

Execution Order:

```
Task A
Task B
Task C
```

Rule

```
Highest Priority READY Task Always Runs
```

This is the most important scheduling rule in FreeRTOS.

Preemptive Scheduling

FreeRTOS supports preemptive scheduling.

When a higher-priority task becomes READY:

```
Running Task
      |
Higher Priority Task Ready
      |
Scheduler Executes
      |
Context Switch
      |
Higher Priority Task Runs
```

The currently running task is interrupted and CPU control is transferred to the higher-priority task.

Advantages

- Fast response time
- Suitable for real-time systems
- Better handling of critical events

Cooperative Scheduling

In cooperative scheduling, tasks voluntarily release the CPU.

The scheduler does not force task switching.

Example:

```
taskYIELD();
```

The running task decides when another task can execute.

Advantages

- Simpler scheduling
- Reduced overhead
- Predictable execution

Disadvantages

- Poor responsiveness if a task never yields.
- Not suitable for most real-time applications.

Round Robin Scheduling

Round Robin scheduling is used when multiple READY tasks have the same priority.

Example:

```
Task A Priority = 3
Task B Priority = 3
Task C Priority = 3
```

Execution:

```
Task A
Task B
Task C
Task A
Task B
Task C
```

Each task receives equal CPU time.

Time Slicing

Time slicing allows tasks of equal priority to share CPU time.

A timer interrupt periodically triggers the scheduler.

Example:

```
Tick = 1 ms
```

Execution:

```
0 ms  -> Task A
1 ms  -> Task B
2 ms  -> Task C
3 ms  -> Task A
```

Configuration:

```
configUSE_TIME_SLICING
```

Context Switching

Context switching is the process of switching CPU execution from one task to another.

Save Current Context

The scheduler saves:

- Program Counter (PC)
- Stack Pointer (SP)
- CPU Registers
- Status Registers

Restore Next Context

The scheduler restores:

- Program Counter
- Stack Pointer
- CPU Registers
- Status Registers

Example:

```
Task A
   |
```

```
Save Context
   |
Scheduler
   |
Restore Context
   |
Task B
```

Scheduler Trigger Events

The scheduler may run when:

System Tick Interrupt Occurs

```
SysTick Interrupt
       |
Scheduler Executes
```

Task Delay Expires

Example:

```
vTaskDelay(100);
```

After timeout:

```
Blocked -> Ready
```

Semaphore Given

Example:

```
xSemaphoreGive();
```

Waiting task becomes READY.

Queue Data Arrives

Example:

```
xQueueSend();
```

Waiting receiver task becomes READY.

Task Notification Received

Example:

```
xTaskNotifyGive();
```

Blocked task becomes READY.

ISR Unblocks Higher Priority Task

Example:

```
Interrupt
    |
ISR
    |
Task Unblocked
    |
Immediate Context Switch
```

Scheduler and System Tick

FreeRTOS scheduler relies on the System Tick interrupt.

Typical configuration:

```
1 Tick = 1 ms
```

Every tick:

1. Tick counter increments.
2. Delays are updated.
3. Blocked tasks are checked.
4. Scheduler decides whether a task switch is required.

Scheduler Configuration

Common FreeRTOS configuration options:

```
configUSE_PREEMPTION
configUSE_TIME_SLICING
configMAX_PRIORITIES
configTICK_RATE_HZ
```

Advantages of FreeRTOS Scheduling

- Deterministic behavior
- Fast task switching
- Priority-based execution
- Efficient CPU utilization
- Suitable for real-time systems
- Supports multitasking

Important Interview Questions

What scheduling algorithm does FreeRTOS use?

FreeRTOS primarily uses priority-based preemptive scheduling with optional round-robin scheduling for equal-priority tasks.

What is the most important scheduling rule in FreeRTOS?

The highest-priority READY task always gets CPU time.

What is time slicing?

Time slicing allows equal-priority tasks to share CPU time fairly.

What triggers the scheduler?

- System Tick Interrupt
- Queue Operations
- Semaphores
- Task Notifications
- Delay Expiration
- ISR Events

What is context switching?

Saving the current task's execution state and restoring another task's state so CPU execution can switch between tasks.

RTOS Scheduling Algorithms

Real-Time Operating Systems use scheduling algorithms to determine which task should execute next while ensuring deadlines are met.

---

1. Rate Monotonic Scheduling (RMS / RMA)

Definition

- Rate Monotonic Algorithm (RMA) is a fixed-priority scheduling algorithm.
- Task priority is assigned according to task period.
- Shorter period ⇒ Higher priority.
- Longer period ⇒ Lower priority.

Priority Assignment

```
Task Period = 10 ms  -> Highest Priority
Task Period = 20 ms
Task Period = 50 ms  -> Lowest Priority
```

Rule:

```
Smaller Period = Higher Priority
```

Example

Task	Execution Time	Period	Priority
T1	2 ms	10 ms	High
T2	4 ms	20 ms	Medium
T3	8 ms	50 ms	Low

Execution Order:

```
T1 > T2 > T3
```

Characteristics

- Fixed priority scheduling.
- Suitable for periodic tasks.
- Easy to implement.
- Widely used in RTOS.
- Priority remains unchanged during execution.

Advantages

- Simple implementation.
- Low scheduling overhead.
- Predictable behavior.
- Easy schedulability analysis.

Disadvantages

- CPU utilization not optimal.
- Lower priority tasks may suffer starvation.
- Not suitable for dynamic workloads.

2. Earliest Deadline First (EDF)

Definition

- EDF is a dynamic-priority scheduling algorithm.
- Task having the nearest (earliest) deadline gets highest priority.
- Priorities change dynamically during runtime.

Priority Assignment

Rule:

```
Earliest Deadline = Highest Priority
```

Example

Task	Deadline
T1	20 ms
T2	10 ms

Task	Deadline
T3	30 ms

Execution Order:

```
T2 -> T1 -> T3
```

because:

```
10 ms < 20 ms < 30 ms
```

Characteristics

- Dynamic priority scheduling.
- Priorities change continuously.
- Suitable for periodic and aperiodic tasks.
- Better CPU utilization than RMS.

Advantages

- Optimal scheduling algorithm for single processor systems.
- Can achieve nearly 100% CPU utilization.
- Better deadline handling.
- Flexible priority assignment.

Disadvantages

- More complex implementation.
- Higher runtime overhead.
- Frequent priority changes.

RMS vs EDF

Feature	RMS (RMA)	EDF
Priority Type	Fixed	Dynamic
Priority Based On	Task Period	Task Deadline
Priority Changes	No	Yes
Complexity	Simple	Complex

Feature	RMS (RMA)	EDF
CPU Utilization	~69% Guaranteed	Up to 100%
Scheduling Overhead	Low	High
Suitable For	Periodic Tasks	Periodic + Aperiodic Tasks
RTOS Usage	Common	Less Common

Quick Interview Answer

What is RMS?

Rate Monotonic Scheduling is a fixed-priority scheduling algorithm where tasks with shorter periods are assigned higher priorities.

What is EDF?

Earliest Deadline First is a dynamic-priority scheduling algorithm where the task with the nearest deadline gets the highest priority.

Which is better?

- RMS is simpler and widely used in RTOS.
- EDF provides better CPU utilization and deadline handling but is more complex to implement.